coursera | UNIVERSITY OF MICHIGAN

Applied Information Extraction in Python

Today's goal • 3/3 items

Module 1
Information Extraction

Module 2
Named Entity Recognition (NER)

Module 3
Neural Network Models

What Is Deep Learning?

Introduction to Deep Learning
Video • 4 min

Understanding Deep Learning: A Shift From Rules to Representation
Reading • 10 min

Knowledge Check: What Is Deep Learning?
Practice Assignment • Grade: 100%

What Are Neural Network Models?

Neural Network Models
Video • 7 min

Activation Functions: How Deep Learning Models Make Decisions
Reading • 10 min

Knowledge Check: What Are Neural Network Models?
Practice Assignment • Grade: 100%

Deeper Neural Networks

Deep Neural Network Models
Video • 6 min

Understanding Deep Neural Network Models: How Depth Enables Learning at Scale
Reading • 10 min

Knowledge Check: Deeper Neural Networks
Practice Assignment • Grade: 100%

How to Build IE Models Using DNNs

Building an Information Extraction Pipeline with BiLSTMs and CRFs
Reading • 10 min

Jupyter Notebook Practice on Training LSTMs
Lab • 1h

Demo: Configuring the Bi-Directional LSTM
Video • 4 min

Module Wrap-Up & Assignment

Assignment 3 Introduction: Building an Information Extraction Pipeline with BiLSTMs and CRFs
Video • 3 min

Build an Information Extraction Pipeline using Deep Neural Networks
Programming Assignment • 3h

Module 3 Assignment
Graded Assignment • 30 min

Module 4
Transformers Transform Information Extraction

Building an Information Extraction Pipeline with BiLSTMs and CRFs

0:26 / 4:32 2x

From Theory to Practice

Over the last few lessons, you've explored how deep neural networks learn complex patterns, from single neurons to stacked layers. Now, it's time to bring it all together into something practical and powerful: a full **Information Extraction (IE)** pipeline.

In this reading, you'll walk through how to build that pipeline using two popular deep learning components:

- **Bi-directional LSTMs (BiLSTMs)**
- **Conditional Random Fields (CRFs)**

This architecture is especially well-suited for **sequence labeling tasks**, like identifying diseases, names, or address components, where **context** matters.

Why Use LSTMs?

LSTMs (Long Short-Term Memory networks) are a type of recurrent neural network that's great at handling sequences like text. Unlike simple feedforward models, LSTMs:

- Remember long-term dependencies
- Capture relationships between distant words
- Avoid forgetting earlier tokens in longer sequences

To go even further, we use **bi-directional LSTMs**, which read the sentence **left-to-right** *and* **right-to-left**. This gives the model a full view of both past and future context.

Step 1: Preprocessing the Data

Before you train a model, your data needs to be structured and encoded. Here's what this looks like in your pipeline:

1.1 Read and Structure the Data

You'll work with labeled sequences:

Tokens: ["University", "of", "Michigan"]

Labels: ["B-ORG", "I-ORG", "L-ORG"]

1.2 Build a Vocabulary

Turn each token into an integer, based on its order of appearance:

{

"<PAD>": 0,

"123": 1,

"Main": 2,

"Street": 3,

"Springfield": 4,

"Illinois": 5,

"62704": 6

}

This also applies to your **label vocabulary**: B-ADDR, L-CITY, etc.

1.3 Convert to Numerical Format

Replace all tokens and labels with their corresponding integers. Be sure to handle unknown tokens (e.g., <UNK>) and padding.

1.4 Pad the Sequences

To ensure consistent input shapes, pad sequences to the same length.

Example:

[2, 3, 4] → [2, 3, 4, 0, 0, 0]

[1, 2, 3] → [1, 2, 3, 0, 0, 0]

Step 2: Designing the Model

Once your data is ready, you can build the model. The architecture typically looks like this:

Input Embedding → BiLSTM Layer → Linear Projection → CRF Layer

2.1 Embedding Layer

- Converts token indices into dense vectors
- Helps the model understand semantic relationships (e.g., "Drive" ≈ "Street")

2.2 BiLSTM Layer

- Processes sequences in both directions
- Learns how each word fits into the broader sentence

2.3 Linear Projection

- Maps the BiLSTM output into label space (e.g., one dimension per label class)

2.4 CRF Layer

- Models label-to-label transitions
- Ensures structured predictions (e.g., B-CITY should be followed by I-CITY, not B-ZIP)

Step 3: Training the Pipeline

Now that your pipeline is built, it's time to train it!

3.1 Loss Function

Most implementations use **negative log-likelihood**, calculated using dynamic programming (often with the Viterbi algorithm).

3.2 Optimizer

Popular options include **Adam** and **SGD**. Learning rate choice can significantly impact performance.

3.3 Training Loop

- Input encoded sequences and labels
- Compute predictions and loss
- Backpropagate the error
- Update weights
- Repeat for several epochs

3.4 Evaluation

You'll evaluate the model at the **sequence level**.

Example:

True: [B-ADDR, I-ADDR, L-ADDR, O, B-ZIP, L-ZIP]

Pred: [B-ADDR, I-ADDR, L-ADDR, O, B-ZIP, L-ZIP]

If even one label is off, the sequence is considered incorrect.

Tips

- **Watch for overfitting**: Use a validation set and consider dropout or early stopping.
- **Handle padding carefully**: Padding tokens should be ignored in both training and evaluation.
- **Keep vocabularies consistent**: Don't retrain your vocab between training and test sets.

Assignment Connection

You'll be implementing this exact architecture in your Module 3 assignment. You'll:

- Reuse your data loading from Module 2
- Build vocabularies
- Pad your sequences
- Train and evaluate a BiLSTM-CRF model

Whenever you feel stuck, come back here, this reading maps directly to your assignment structure.

Suggested Resources

- **https://github.com/createmomo/CRF-Layer-on-the-Top-of-BiLSTM**
- **Understanding LSTMs – Colah's Blog**

Reflection Prompt

We're combining **BiLSTMs** and **CRFs**, but why both?

What role does each component play in making predictions more accurate? Could the pipeline still work with just one of them? Why might that be limiting?

Go to next item Completed

Like Dislike Report an issue